



Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering

Experiment No: 12

Academic Year: 2025-2026

Year: TY (A, B, C)

Semester: II

Course: System Verilog for Verification Lab

Course Code: BET26PE12

Title:

Design and Verify a FIFO (First-In-First-Out) Memory Using System Verilog Testbench

Objectives of the Expt.:

1. To understand the architecture and working principle of FIFO memory.
2. To design a parameterized synchronous FIFO using SystemVerilog.
3. To write a SystemVerilog testbench to verify FIFO operations including write, read, full, and empty conditions.
4. To simulate the FIFO design and analyze the output waveforms.

CO and PO MAPPED:

CO1,CO2,PO1,PO2

Equipment / Software Required:

Open Source: EDA Playground

Simulator: Icarus Verilog

Theory:

1. Introduction to FIFO Memory

A FIFO (First-In-First-Out) is a type of data buffer or queue in which the first data word written into the memory is also the first word to be read out. This is analogous to a queue at a ticket counter where the person who arrives first is served first. FIFO buffers are fundamental components in digital systems design and are widely used in communication interfaces, data streaming pipelines, processor design, and synchronization between clock domains.

The core behavioral characteristic of a FIFO is that data ordering is strictly preserved. Unlike a random-access memory (RAM) where any address can be read at will, a FIFO enforces sequential access. Data must be consumed in the exact order it was produced. This property makes FIFO ideal for applications where data integrity and ordering are critical.

2. Types of FIFO

2.1 Synchronous FIFO: In a synchronous FIFO, both the write and read operations are governed by the same clock signal. All state changes including pointer updates, memory writes, and status flag generation occur on the same clock edge. This simplifies design and verification because there is no need to handle metastability or clock domain crossing issues.

2.2 Asynchronous FIFO: In an asynchronous FIFO, the write port and read port operate under separate independent clock domains. This is used in systems where a producer (e.g., a high-speed serial receiver) and a consumer (e.g., a processor bus) operate at different frequencies. Designing an asynchronous FIFO requires careful gray-code pointer synchronization to avoid metastability errors. In this experiment, we design and verify a Synchronous FIFO using SystemVerilog.

3. FIFO Architecture and Internal Structure

A typical synchronous FIFO consists of the following functional blocks:

- (a) Memory Array: A 2D register array `mem[DEPTH][WIDTH]` that stores the actual data words. The depth defines how many data words can be stored at one time. The width defines how many bits constitute a single data word.
- (b) Write Pointer (`wr_ptr`): A counter that holds the address of the next available write location. After each successful write operation (when `wr_en` is asserted and the FIFO is not full), the write pointer is incremented. When it reaches `DEPTH`, it wraps around to 0, creating a circular buffer.
- (c) Read Pointer (`rd_ptr`): A counter that holds the address of the next valid read location. After each successful read (when `rd_en` is asserted and the FIFO is not empty), the read pointer is incremented. It also wraps around circularly.
- (d) Occupancy Counter (`count`): Tracks the number of valid data words currently stored in the FIFO. It increments on write, decrements on read, and remains unchanged on simultaneous read+write.
- (e) Status Flags:
 - `full = (count == DEPTH)` -- FIFO is full; no further writes allowed.
 - `empty = (count == 0)` -- FIFO is empty; no reads allowed.
- (f) Control Logic: Generates enable signals for the memory array, manages pointer arithmetic, and updates the count register based on `wr_en` and `rd_en` inputs.

4. FIFO Block Diagram

The block diagram below shows the top-level architecture of the synchronous FIFO, with the Write Port (`wr_clk`, `wr_en`, `wr_data`, `full`), internal Memory Array, and Read Port (`rd_clk`, `rd_en`, `rd_data`, `empty`).

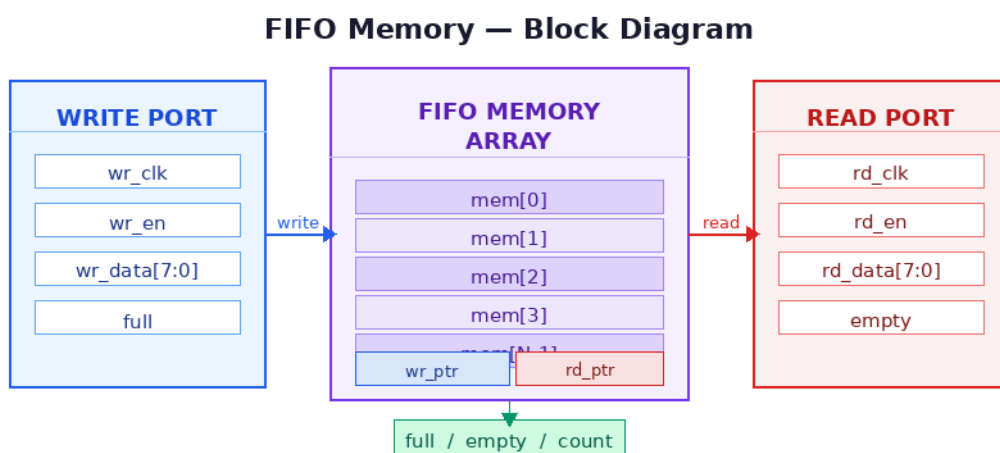


Fig. 1 -- FIFO Memory Block Diagram showing Write Port, Memory Array, and Read Port

5. FIFO Operation

5.1 Write Operation: A write occurs when $wr_en = 1$ AND $full = 0$. On the rising clock edge, wr_data is stored at $mem[wr_ptr]$, then wr_ptr is incremented modulo DEPTH, and count is incremented by 1.

5.2 Read Operation: A read occurs when $rd_en = 1$ AND $empty = 0$. On the rising clock edge, $mem[rd_ptr]$ is driven to rd_data , rd_ptr is incremented modulo DEPTH, and count is decremented by 1.

5.3 Simultaneous Read and Write: If both wr_en and rd_en are asserted simultaneously and the FIFO is neither full nor empty, both operations execute in the same clock cycle and count remains unchanged.

5.4 Full Condition: $count == DEPTH$. Any wr_en assertion is ignored. The full flag deasserts only after a read decrements count below DEPTH.

5.5 Empty Condition: $count == 0$. Any rd_en assertion is ignored. The empty flag deasserts only after a write increments count above 0.

6. Timing Diagram

The timing diagram below illustrates a sample sequence of write and read operations, showing wr_data values D0 to D4 being written while wr_en is high, the full flag asserting after DEPTH writes, and rd_data appearing one cycle after rd_en is asserted.

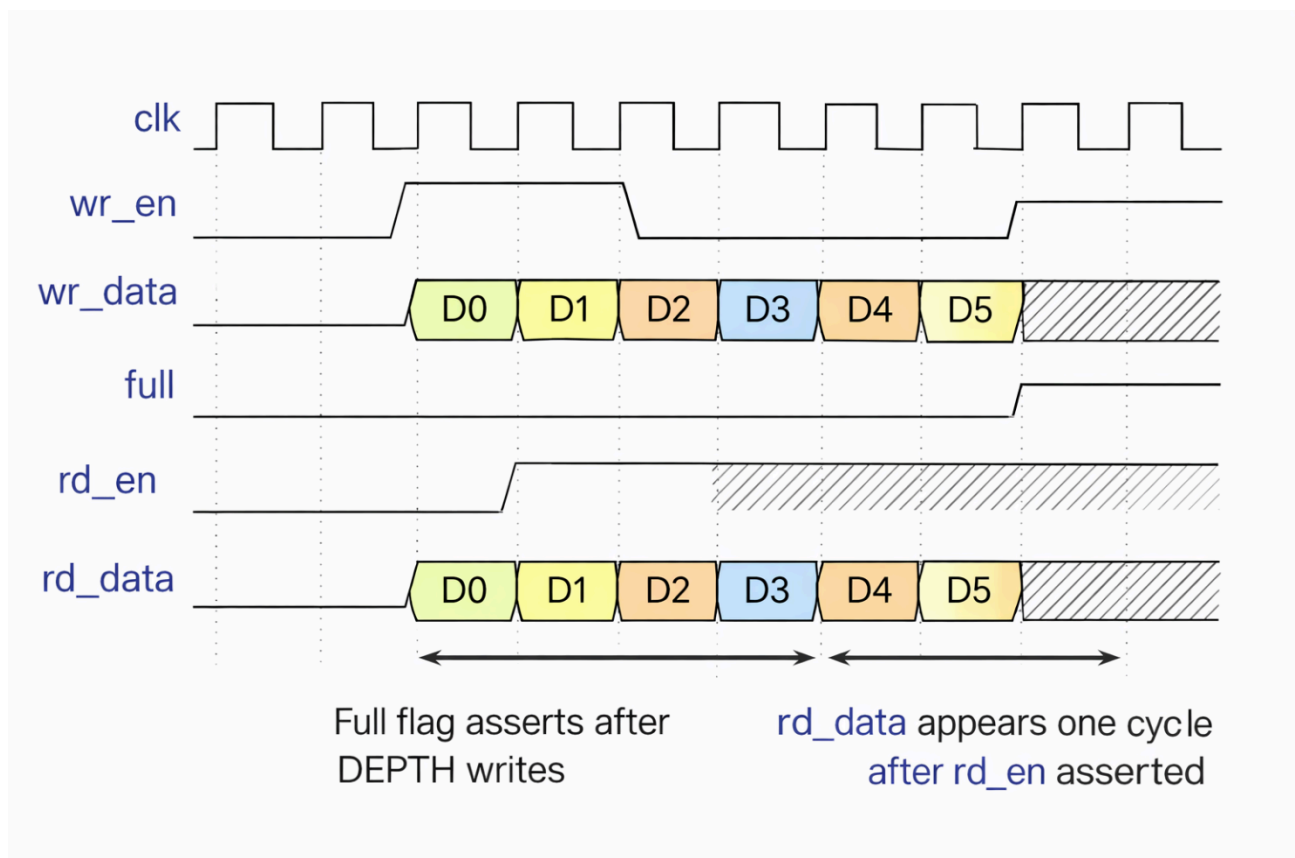


Fig. 2 -- FIFO Read/Write Timing Diagram

7. Circular Buffer Concept

Internally, the FIFO uses a circular buffer (ring buffer) to manage its memory. The write and read pointers advance through the memory array and wrap around when they reach DEPTH-1. This eliminates the need to physically shift data, resulting in $O(1)$ time complexity for both read and write operations regardless of FIFO depth.

The FIFO is considered full when write pointer catches up to read pointer after wrapping, and empty when both pointers are at the same position. Using a separate count register (as in this design) avoids ambiguity between full and empty conditions.

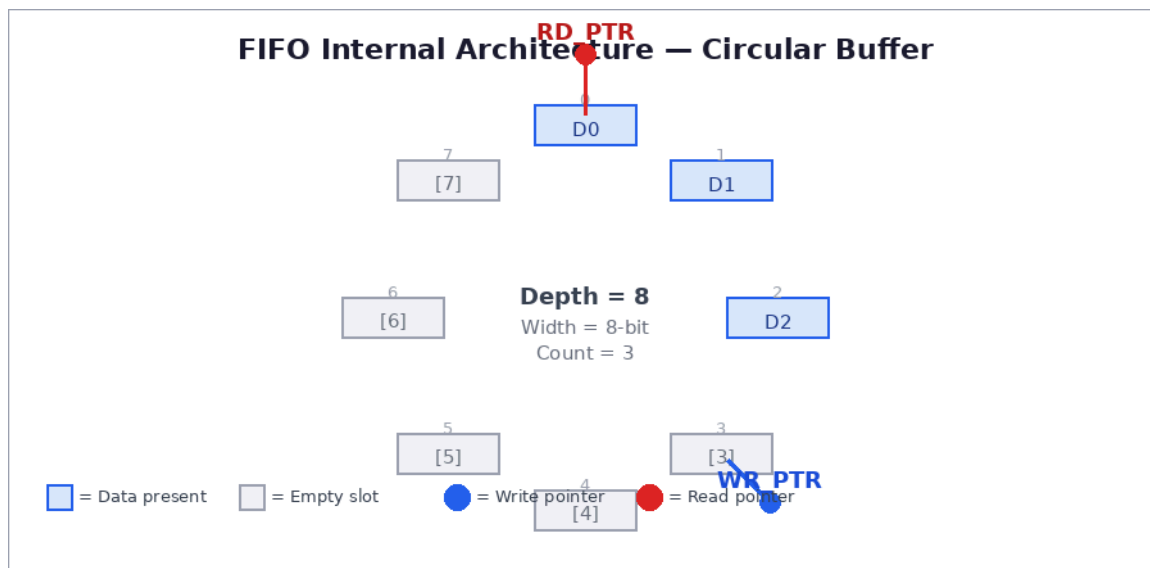


Fig. 3 -- FIFO Circular Buffer Architecture (Depth=8, 3 slots occupied, blue=data present)

The write pointer indicates the next location for writing, while the read pointer indicates the next location to be read. The buffer uses a circular structure where pointers wrap around after reaching the last index.

Full condition occurs when write pointer catches read pointer after wrap-around, and empty condition occurs when both pointers are equal.

8. Applications of FIFO Memory

FIFO buffers are used extensively in modern digital systems. Key applications include:

- (a) UART TX/RX buffering to handle burst data at varying baud rates.
- (b) Clock Domain Crossing (CDC) -- safely transferring data between two different clock domains using asynchronous FIFO.
- (c) CPU instruction pipeline and data cache buffering to absorb latency differences.
- (d) Network packet buffering in switches and routers.
- (e) USB, PCIe, and DDR memory controller interfaces for data queuing.
- (f) Audio/Video streaming buffers to absorb burst data and regulate steady output.

9. FIFO Design Parameters

DEPTH: Number of data words the FIFO can hold simultaneously. Parameterizing depth allows reuse across designs. Common values: 16, 32, 64, 256.

WIDTH: Number of bits in each data word. Common values: 8 (byte), 16, 32. Parameterizing width allows the same RTL module to work as a byte FIFO or word FIFO without redesign.

Latency: Synchronous FIFOs have one-clock-cycle read latency. Data written in cycle N appears on rd_data in cycle N+1 (registered output).

SystemVerilog FIFO Design Code (fifo.sv):

```
module fifo #(
    parameter DEPTH = 8,
    parameter WIDTH = 8
) (
    input  logic          clk,
    input  logic          rst_n,
    input  logic          wr_en,
    input  logic          rd_en,
    input  logic [WIDTH-1:0] wr_data,
    output logic [WIDTH-1:0] rd_data,
    output logic          full,
    output logic          empty
);

    // Internal memory array
    logic [WIDTH-1:0] mem [0:DEPTH-1];

    // Write and read pointers
    logic [$clog2(DEPTH)-1:0] wr_ptr, rd_ptr;

    // Occupancy counter
    logic [$clog2(DEPTH):0] count;

    // Status flags (combinational)
    assign full  = (count == DEPTH);
    assign empty = (count == 0);

    // Write logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) wr_ptr <= 0;
        else if (wr_en && !full) begin
            mem[wr_ptr] <= wr_data;
            wr_ptr      <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
        end
    end

    // Read logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rd_ptr <= 0;
            rd_data <= 0;
        end else if (rd_en && !empty) begin
            rd_data <= mem[rd_ptr];
            rd_ptr  <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
        end
    end

    // Count logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) count <= 0;
        else begin
            case ({wr_en & !full, rd_en & !empty})
                2'b10: count <= count + 1; // write only
                2'b01: count <= count - 1; // read only
                default: count <= count;    // no change or simultaneous
            endcase
        end
    end
```

```

end

endmodule

```

SystemVerilog Testbench (fifo_tb.sv):

```

`timescale 1ns/1ps
module fifo_tb;
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    logic          clk, rst_n;
    logic          wr_en, rd_en;
    logic [WIDTH-1:0] wr_data;
    logic [WIDTH-1:0] rd_data;
    logic          full, empty;

    // Instantiate DUT (Device Under Test)
    fifo #(.DEPTH(DEPTH), .WIDTH(WIDTH)) dut (
        .clk(clk), .rst_n(rst_n),
        .wr_en(wr_en), .rd_en(rd_en),
        .wr_data(wr_data), .rd_data(rd_data),
        .full(full), .empty(empty)
    );

    // Clock generation: 10 ns period (100 MHz)
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        // Apply reset
        rst_n = 0; wr_en = 0; rd_en = 0; wr_data = 0;
        @(posedge clk); @(posedge clk);
        rst_n = 1;

        // Test 1: Write DEPTH words to fill FIFO
        $display("--- Test 1: Fill FIFO ---");
        repeat (DEPTH) begin
            @(posedge clk);
            wr_en = 1;
            wr_data = $random;
            $display("WRITE: data=%0h full=%b empty=%b", wr_data, full, empty);
        end
        @(posedge clk); wr_en = 0;
        $display("Full flag = %b (expected 1)", full);

        // Test 2: Write to full FIFO (should be ignored)
        $display("--- Test 2: Overflow attempt ---");
        @(posedge clk); wr_en = 1; wr_data = 8'hFF;
        @(posedge clk); wr_en = 0;

        // Test 3: Read all data out
        $display("--- Test 3: Empty FIFO ---");
        repeat (DEPTH) begin
            @(posedge clk); rd_en = 1;
            @(posedge clk);
            $display("READ : data=%0h full=%b empty=%b", rd_data, full, empty);
        end
        @(posedge clk); rd_en = 0;
        $display("Empty flag = %b (expected 1)", empty);

        // Test 4: Simultaneous read and write
        $display("--- Test 4: Simultaneous R+W ---");

```

```

    @(posedge clk); wr_en = 1; wr_data = 8'hAA;
    @(posedge clk); wr_en = 1; rd_en = 1; wr_data = 8'hBB;
    @(posedge clk); wr_en = 0; rd_en = 1;
    @(posedge clk); rd_en = 0;

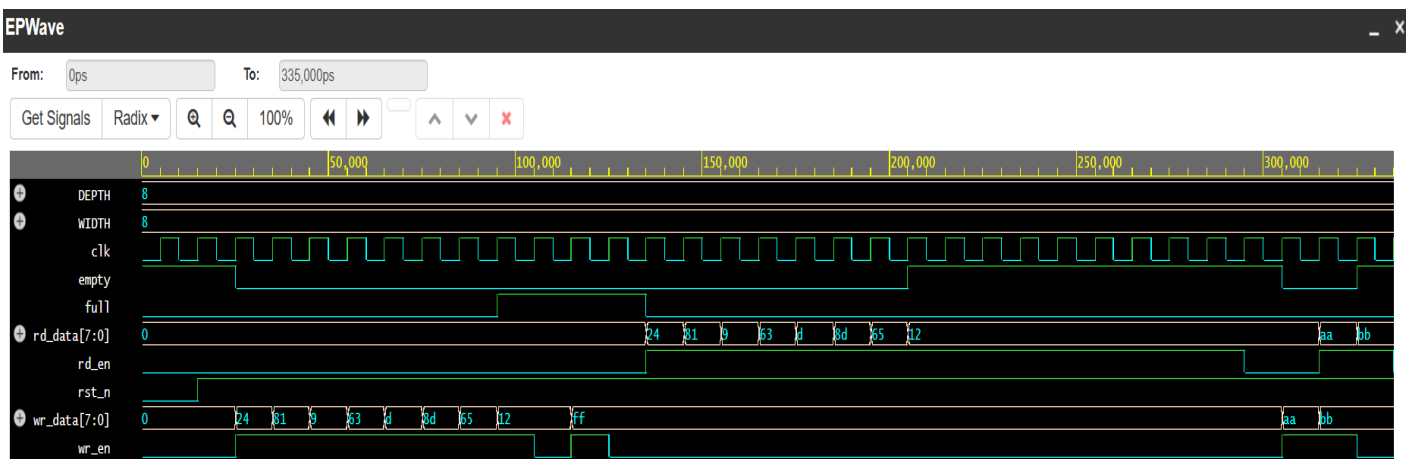
    $display("--- Simulation Complete ---");
    $finish;
end

// VCD waveform dump
initial begin
    $dumpfile("fifo_tb.vcd");
    $dumpvars(0, fifo_tb);
end

endmodule

```

Output Waveform / Simulation Results:



After running simulation in EDA Playground, verify the following in waveform window:

- wr_data values appear in memory in the exact order they were written.
- rd_data follows the same order one clock cycle after rd_en is asserted (registered output latency).
- full flag asserts exactly when DEPTH consecutive writes have occurred without any reads.
- empty flag asserts after all stored data words have been read out.
- No data corruption or ordering violation occurs during simultaneous read+write operations.

Conclusion:

In this experiment, a parameterized synchronous FIFO memory was successfully designed using SystemVerilog. The design employs a circular buffer with independent write and read pointer registers and a dedicated occupancy counter to track the number of valid data words. Full and empty status flags are generated combinatorially from the count register. A comprehensive SystemVerilog testbench was developed and executed to verify: (1) correct sequential data storage and retrieval in First-In-First-Out order, (2) proper assertion of the full flag when DEPTH locations are occupied, (3) proper assertion of the empty flag when no valid data remains, (4) rejection of write operations when full and read operations when empty, and (5) correct behavior during simultaneous read and write. Simulation waveforms confirmed that the FIFO operates correctly

across all test scenarios, demonstrating the effectiveness of SystemVerilog for both RTL design and functional verification.

Answer the following questions:

- 1. What is the difference between a synchronous and asynchronous FIFO? In which applications is each preferred?**
- 2. How does a FIFO differ from a stack (LIFO)? Give one application of each.**
- 3. What would happen if only pointer comparison is used to detect full/empty conditions?**
- 4. What is the significance of the \$clog2 system function used in the design?**
- 5. What changes would be required to convert this synchronous FIFO into an asynchronous FIFO?**